



A modern Django-based music platform for uploading tracks, importing YouTube audio safely, and building a personal favorites library.

Built in approximately **1 month** as a focused personal development project.

English Version



Table of Contents

1. Project Overview
2. Features
3. Technologies Used
4. Project Architecture
5. UI / Design
6. Installation Guide
7. Project Structure
8. Security
9. API and Data Interfaces
10. Future Improvements
11. Developer
12. Vision
13. Support the Project
14. Қазақша Нұсқа



Project Overview

QazSound is a Django web platform designed for publishing, exploring, and saving music tracks in a clean and modern interface. It combines traditional upload-based audio publishing with YouTube-based streaming workflows, giving creators flexibility and listeners a simple, engaging discovery experience.

This project was developed in approximately **one month** by a university student as an intensive personal learning project focused on real-world full-stack engineering skills. Instead of building a purely static portfolio project, the goal was to build something operational: user authentication, media handling, dynamic interaction logic, admin analytics, frontend state management, and deployment-ready settings.

What the website does

- Allows creators to upload local audio files with metadata and cover images.
- Allows creators to publish YouTube-based tracks by storing YouTube metadata and streaming audio on demand without server-side downloading.
- Provides track discovery views: featured tracks, fresh uploads, and trending tracks.
- Supports user authentication, profile settings, language preference, and avatar management.
- Enables engagement through likes/favorites, including AJAX-powered interactions.
- Includes a global persistent audio player with playback history and local state restoration.

What problem it solves

QazSound addresses a full-platform challenge, not only CRUD pages: user content publishing, listener discovery, real-time interaction, admin oversight, and deploy-ready configuration in one coherent system. It also unifies uploaded tracks and YouTube-source tracks in a single listening workflow.

Who it is for

- **Music creators** who want a simple way to publish tracks and metadata.
- **Listeners** who want a modern, card-based browsing and favorites experience.
- **Students and junior developers** who want to study a practical Django architecture with services/selectors separation.
- **Open-source collaborators** who want a foundation for further feature work (playlists, analytics, APIs, mobile support).

Why it was created

QazSound was created as a learning journey with strict engineering goals:

- turn classroom concepts into production-like implementation;
- practice Django architecture beyond basic tutorials;
- implement both backend validation and frontend interaction patterns;
- learn deployment preparation (static files, env config, WSGI server, database switching).

What makes it unique

- Hybrid content model: local uploads + YouTube streaming.
- Domain-safe YouTube URL normalization and metadata extraction.
- Custom global audio player with history and persistent state in localStorage.
- AJAX likes synchronized across cards, detail pages, and player controls.

- Custom Django admin dashboard with analytics cards and chart visualizations.
 - Bilingual-oriented foundation (Kazakh and Russian interface support in settings).
-



Features

User Features

- **Account system** with registration, login, logout, and profile management.
- **Track publishing workflow** with source selection (`UPLOAD` or `YOUTUBE`).
- **Metadata enrichment** from YouTube via preview endpoint and on-demand fetch.
- **Favorites system** with instant like/unlike behavior via AJAX and fallback form submit.
- **Global player bar** with play/pause, previous/next history navigation, and favorite toggle.
- **Persistent playback state** (track, position, volume, playback intent) saved in browser storage.
- **Search and filtering** by query and genre.
- **Personal profile page** with user identity block and “my tracks” section.
- **Language preference** selection stored in profile settings.

Admin Features

- Custom Django admin branding and dark UI (Jazzmin-based enhancements).
- Track moderation with rich list views, cover previews, and media/source metadata.
- Bulk admin actions:
 - mark selected tracks as featured,
 - reset `plays_count` for selected tracks.
- Artist protection rules to prevent accidental deletion when tracks still exist.
- Dashboard analytics:
 - total users,
 - total tracks,
 - total likes,
 - total artists,
 - 7-day upload trend chart,
 - top liked tracks chart and quick links.

Website / Platform Features

- Responsive card-based layout optimized for mobile and desktop.
 - Dynamic partial-page navigation for eligible links/forms using Fetch + History API.
 - Safe fallback behaviors when JavaScript fails.
 - Structured app organization (`tracks` , `users` , `interactions`) with clear separation of concerns.
 - Deployment-ready static file handling through WhiteNoise.
 - Optional PostgreSQL support via `DATABASE_URL` .
 - API endpoints for track list and track detail serialization.
-

🔧 Technologies Used

This project intentionally uses a pragmatic stack centered around Django fundamentals and web standards.

Technology	Why it was used
Python	Core language for backend logic, validation, service orchestration, and maintainable business rules.
Django	Provides robust ORM, admin panel, auth system, forms, middleware, and secure defaults for rapid but structured development.
HTML5	Semantic template structure for reusable components, accessible forms, and content hierarchy.
CSS3	Custom visual system with variables, responsive breakpoints, gradients, card styles, and animation effects.
JavaScript (Vanilla)	Dynamic UX layer for global player logic, AJAX likes, metadata fetch, and navigation without full reloads.
Bootstrap / Tailwind	Not used intentionally ; custom CSS was chosen to practice design system building from scratch.
SQLite	Default local development database for quick startup and simple setup.
PostgreSQL	Production-ready option supported via <code>dj-database-url</code> + <code>psycopg2-binary</code> through environment configuration.
Pillow	Image processing support for user avatars and track cover uploads.
yt-dlp	Safe extraction of YouTube metadata and best-audio stream URL for supported YouTube tracks.
WhiteNoise	

Technology	Why it was used
	Efficient serving of static assets in production deployments without external static service complexity.
Gunicorn	WSGI application server used for production hosting (configured via Procfile).
Git / GitHub	Version control, collaborative workflows, transparent project history, and open-source distribution.
VS Code	Main IDE for coding, refactoring, debugging, and project navigation during the 1-month build cycle.

This stack balances learning depth and production realism: fast Django development, custom frontend control, and deploy-ready infrastructure.



Project Architecture

QazSound uses a modular Django architecture with app-level boundaries and a service-selector pattern for clarity.

High-level layout

- `config/` handles global settings, URL dispatching, WSGI/ASGI, and test stubs.
- `apps/tracks/` owns track domain models, forms, services, selectors, API views, and YouTube utilities.
- `apps/users/` owns registration/login/profile/settings and profile extension model.
- `apps/interactions/` owns likes, favorites, and playlist-related data models.
- `templates/` contains reusable UI templates and app-specific pages.
- `static/` contains CSS, JavaScript, admin customization, and placeholder assets.
- `media/` stores uploaded files (audio, covers, avatars) in development/local environments.

Internal flow (request lifecycle summary)

1. Browser requests route.
2. URL dispatcher sends request to app view.
3. View uses selectors (reads) and services (writes/business rules).
4. Forms validate user input.
5. Templates render HTML.
6. JavaScript enhances UX (likes, player, metadata fetch, navigation).

Domain model overview

- **Artist**: stores artist metadata and links to tracks.
- **Genre**: unique genre taxonomy with auto-generated slug.
- **Track**: central content entity supporting:
 - source type (`UPLOAD` / `YOUTUBE`),
 - media fields,
 - YouTube identifiers,
 - likes count via relation,
 - play count field,
 - ownership relation to uploader.
- **Like**: unique (`user`, `track`) relation preventing duplicate likes.
- **Playlist / PlaylistItem**: foundational schema for playlist extension.
- **UserProfile**: extends auth user with display name, avatar, and preferred language.

Services and selectors pattern

QazSound separates logic into:

- **selectors** for optimized read queries (`select_related`, `prefetch_related`, aggregates),
- **services** for write/business logic (track creation, source handling, YouTube extraction orchestration),
- **forms** for input-level validation and user-facing error messaging.

This structure keeps views lean and easier to maintain.

YouTube source handling

The YouTube flow is intentionally strict:

- validates allowed domains (`youtube.com`, `youtu.be` variants),
- extracts and normalizes canonical video URL,
- fetches metadata (title, author, thumbnail, duration),
- stores only needed metadata,
- resolves stream URL on demand via `yt-dlp` extractor when playback is requested.

This design avoids permanent media duplication on server storage.

Frontend interaction architecture

- `likes.js`: asynchronous like toggling with DOM state sync.
- `player.js`: global playback engine, history stack, persistence, and cross-component state updates.

- `forms.js`: track form source toggling and YouTube metadata fetch helper.
 - `navigation.js`: enhanced same-origin navigation for eligible links/forms.
 - `main.js`: toast lifecycle and small UI helpers.
-

UI / Design

QazSound follows a modern, creator-focused visual direction with a dark neon palette and glassy card components.

Design philosophy

- **Clean UI**: clear spacing, readable typography, focused content hierarchy.
- **Modern layout**: hero section, split content blocks, cards, badges, and sticky navigation.
- **Responsive interface**: breakpoints for mobile-first usability and desktop scalability.
- **Card-based design**: tracks are represented as reusable visual cards with quick actions.
- **Dark/light aesthetic influence**: core implementation is dark-themed with accent gradients and ambient backgrounds.
- **User-friendly navigation**: compact menu, direct search, and clearly grouped actions.

Visual identity details

- CSS variables in `theme.css` define semantic colors (`--accent`, `--surface`, `--muted`, etc.).
- Animated ambient gradient overlay adds depth without heavy assets.
- Track cards use hover motion, soft borders, and contextual source badges.
- Player bar remains consistent across pages, creating continuity in listening flow.

UX decisions worth noting

- Like state is mirrored in all relevant locations (cards, detail, player) to avoid confusion.
 - YouTube metadata fetch reduces manual typing and metadata inconsistency.
 - Empty-state components keep low-content pages informative and actionable.
 - Form validation combines client-side hints and backend enforcement for better trust.
-

⚙️ Installation Guide

Prerequisites

- Python 3.10+ recommended
- `pip`
- Git
- Optional: virtual environment tool (`venv` is built-in)

1. Clone repository

```
git clone <your-repository-url>
cd QazSound
```

2. Create and activate virtual environment

```
python -m venv venv
```

macOS/Linux:

```
source venv/bin/activate
```

Windows (PowerShell):

```
venv\Scripts\Activate.ps1
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Configure environment variables

Copy `.env.example` to `.env` and edit values:

```
cp .env.example .env
```

Recommended minimum for local:


```
SECRET_KEY=django-insecure-change-me
DEBUG=True
ALLOWED_HOSTS=127.0.0.1,localhost
ENABLE_YTDLP_YOUTUBE_STREAM=True
```

For PostgreSQL + Firebase media storage:

```
DATABASE_URL=postgresql://postgres:change-me@localhost:5432/qazsound
FIREBASE_STORAGE_BUCKET=your-bucket-name
FIREBASE_PROJECT_ID=your-gcp-project-id
FIREBASE_CREDENTIALS_PATH=firebase-service-account.json
FIREBASE_MEDIA_LOCATION=media
FIREBASE_QUERYSTRING_AUTH=True
```

5. Apply migrations

```
python manage.py migrate
```

6. (Optional) Create admin user

```
python manage.py createsuperuser
```

7. Run development server

```
python manage.py runserver
```

Open: <http://127.0.0.1:8000/>

8. Collect static files (for production-like runs)

```
python manage.py collectstatic --no-input
```

9. Run test suite

```
python manage.py test
```

Environment variables reference

Variable	Purpose
<code>SECRET_KEY</code>	Django cryptographic signing key.
<code>DEBUG</code>	Enables/disables debug mode.
<code>ALLOWED_HOSTS</code>	Allowed hostnames for requests.
<code>CSRF_TRUSTED_ORIGINS</code>	Trusted origins for CSRF-protected requests.
<code>DATABASE_URL</code>	Enables PostgreSQL/other DB connection via URL.
<code>DB_CONN_MAX_AGE</code>	DB persistent connection tuning.
<code>FIREBASE_STORAGE_BUCKET</code>	Google Cloud Storage bucket used by Firebase Storage for uploaded media.
<code>FIREBASE_PROJECT_ID</code>	Optional Google Cloud/Firebase project id for the storage client.
<code>FIREBASE_CREDENTIALS_PATH</code>	Relative or absolute path to a Firebase service account JSON file.
<code>FIREBASE_CREDENTIALS_JSON</code>	Inline Firebase service account JSON for platforms that only support env vars.
<code>FIREBASE_MEDIA_LOCATION</code>	Bucket subfolder prefix for uploaded media files.
<code>FIREBASE_QUERYSTRING_AUTH</code>	Sign media URLs for private buckets; set <code>False</code> only for public media buckets.
<code>FIREBASE_URL_EXPIRATION_SECONDS</code>	Signed media URL lifetime in seconds.
<code>FIREBASE_FILE_OVERWRITE</code>	Keep unique filenames in storage when set to <code>False</code> .
<code>FIREBASE_BLOB_CHUNK_SIZE</code>	Optional resumable upload chunk size in bytes for large audio files.
<code>SECURE_SSL_REDIRECT</code>	Force HTTPS redirection in production.
<code>SESSION_COOKIE_SECURE</code>	Secure-only session cookies.
<code>CSRF_COOKIE_SECURE</code>	Secure-only CSRF cookies.
<code>ENABLE_YTDLP_YOUTUBE_STREAM</code>	Toggle YouTube stream extraction feature.
<code>YTDLP_STREAM_FORMAT</code>	Preferred <code>yt-dlp</code> format selector.
<code>YTDLP_REQUEST_TIMEOUT_SECONDS</code>	Request timeout for extraction operations.

Deployment note

This repository includes:

- `build.sh`: dependency install, migrate, collectstatic
- `Procfile`: Gunicorn startup command

These files align with common PaaS workflows (for example, Render-style deployment).

Project Structure

```
QazSound/
├─ manage.py
├─ requirements.txt
├─ Procfile
├─ build.sh
├─ db.sqlite3
├─ config/
│   ├─ settings.py
│   ├─ urls.py
│   ├─ wsgi.py
│   ├─ asgi.py
│   └─ tests/
├─ apps/
│   ├─ tracks/
│   │   ├─ models.py
│   │   ├─ views.py
│   │   ├─ forms.py
│   │   ├─ services.py
│   │   ├─ selectors.py
│   │   ├─ api.py
│   │   ├─ downloader.py
│   │   ├─ utils.py
│   │   └─ urls.py
│   ├─ users/
│   │   ├─ models.py
│   │   ├─ forms.py
│   │   ├─ views.py
│   │   ├─ signals.py
│   │   └─ urls.py
│   └─ interactions/
│       ├─ models.py
│       ├─ views.py
│       ├─ services.py
│       ├─ selectors.py
│       └─ urls.py
├─ templates/
│   ├─ base.html
│   ├─ admin/
│   ├─ tracks/
│   ├─ users/
│   ├─ interactions/
│   ├─ components/
│   └─ includes/
├─ static/
```

```
| ├── css/
| ├── js/
| ├── img/
| ├── admin/
└── media/
```

Folder explanation

- `config/` : global Django configuration and root URL wiring.
- `apps/tracks/` : track catalog domain, source handling, YouTube logic, and JSON API.
- `apps/users/` : authentication-adjacent UI, profile model, language preference handling.
- `apps/interactions/` : likes/favorites and playlist data foundations.
- `templates/` : server-rendered pages and reusable components.
- `static/` : style system, client JS behavior, and placeholders.
- `media/` : uploaded user assets during runtime.



Security

QazSound relies on Django security fundamentals and adds domain-specific safeguards.

Built-in Django security usage

- CSRF middleware enabled globally.
- Authentication middleware for user identity/session management.
- Password validators enabled (length, common password, numeric password checks, etc.).
- Clickjacking protection through `XFrameOptionsMiddleware`.
- Secure cookie and HTTPS toggles configurable via environment variables.

CSRF and request safety

- All sensitive forms include `{% csrf_token %}`.
- AJAX requests (likes/player toggle) send CSRF token headers.
- Unsafe redirects are blocked using `url_has_allowed_host_and_scheme` in next-URL resolution.

Authentication and authorization

- `@login_required` protects content creation, editing, deleting, favorites, profile settings, and like actions.
- Track ownership checks ensure only owners can edit/delete their tracks.

- Admin features remain protected by Django admin permission model.

Input validation and media constraints

- Audio extension and MIME checks limit uploads to expected audio formats.
- Cover image type and size checks prevent invalid file submissions.
- YouTube URLs are validated against explicit allowed host list and strict ID pattern.
- Domain normalization reduces malformed-link and edge-case inconsistencies.

Database-level integrity

- Unique constraint on likes (`user` , `track`) prevents duplicate engagement records.
- Unique genre names/slugs ensure clean taxonomy.
- Structured relationships enforce data consistency across tracks, artists, profiles, and interactions.



API and Data Interfaces

QazSound includes lightweight JSON endpoints for integration and testing.

Endpoints

Endpoint	Method	Description
<code>/api/tracks/</code>	GET	Returns filtered track list (query + genre support).
<code>/api/tracks/<id>/</code>	GET	Returns detailed track payload.
<code>/tracks/youtube/metadata/</code>	GET	Returns metadata preview for a YouTube URL.
<code>/tracks/<id>/stream/</code>	GET	Resolves and redirects (or returns JSON) with YouTube audio stream URL.

API output highlights

Track JSON contains:

- `id` , `title` , `artist`
- `genres`
- `cover_url` , `audio_url`
- `likes_count` , `plays_count`
- `source_type` , `youtube_url` , `youtube_id`
- `is_liked` (for authenticated requests)
- `created_at`

Important behavior notes

- YouTube stream URLs are resolved on demand and may expire depending on upstream media rules.
 - `plays_count` exists as a tracked field and ranking input; automated increment workflows can be extended further.
-



Future Improvements

Planned improvements to move QazSound toward a broader production roadmap:

1. **Docker support**

Containerized development and deployment for reproducible environments.

2. **Expanded API integration**

Add richer API layer for mobile/third-party clients.

3. **Cloud media storage**

Integrate scalable object storage for assets.

4. **Playback analytics pipeline**

Add event tracking and automated play metrics.

5. **Test coverage expansion**

Add integration tests for forms, permissions, and API contracts.

6. **CI/CD automation**

Add lint/test/deploy verification pipelines.



Developer

Turginbay Bekzat

2nd Year Student

Satbayev University, Kazakhstan

Future Full Stack Developer

Developer story

QazSound was built in one month as a focused engineering challenge and a practical transition from isolated exercises to a complete product lifecycle. The project combines backend architecture, frontend interaction design, deployment preparation, and technical documentation into one portfolio-level implementation.

Vision

The long-term vision behind QazSound is grounded in four principles:

1. **Learning by building**

Real progress comes from shipping complete systems, not isolated snippets.

2. **Innovation through iteration**

Start with a strong foundation, then improve through measurable refinements.

3. **Open-source contribution**

Share work transparently, invite feedback, and grow through collaboration.

4. **Real-world software mindset**

Build with scalability, maintainability, and user trust in mind from day one.

QazSound is intended to evolve from a student project into a mature open-source platform that demonstrates practical Django engineering in a culturally meaningful context.

Support the Project

If this project is useful or interesting, you can support it by:

-  **Starring** the repository
 -  **Forking** and building your own version
 -  **Sharing** it with students, developers, and music communities
 -  **Contributing** improvements (features, tests, docs, UX fixes)
-

Қазақша нұсқа



Мазмұны

1. Жоба туралы
2. Мүмкіндіктер
3. Қолданылған технологиялар
4. Жоба архитектурасы
5. UI / Дизайн
6. Орнату нұсқаулығы
7. Жоба құрылымы
8. Қауіпсіздік
9. API және деректер интерфейстері

10. Болашақ жетілдірулер
 11. Өзірлеуші
 12. Көзқарас
 13. Жобаны қолдау
-

Жоба туралы

QazSound - бұл Django негізінде жасалған заманауи музыкалық веб-платформа. Жоба пайдаланушыларға трек жариялауға, YouTube сілтемесінен аудио ағынды қауіпсіз түрде қолдануға, сүйікті тректерін жинауға және ыңғайлы интерфейс арқылы контентті зерттеуге мүмкіндік береді.

Бұл жоба шамамен **1 ай** ішінде университет студентінің жеке даму мақсатындағы интенсивті инженерлік тәжірибесі ретінде жасалған. Мақсат тек "қарапайым студенттік сайт" құру емес, нақты өндірістік ортаға жақын архитектураны практикада іске асыру болды: аутентификация, медиа өңдеу, формаларды тексеру, динамикалық интерфейс, әкімшілік аналитика, deploy-ға дайын конфигурация.

Веб-сайт не істейді?

- Авторларға локалды аудио файлдарды метадеректермен бірге жүктеп жариялауға мүмкіндік береді.
- YouTube сілтемесін енгізіп, бейне метадерекін алып, аудионы сұраныс кезінде ойнатады.
- Тректерді "Featured", "Fresh uploads", "Trending" блоктары арқылы ұсынады.
- Пайдаланушы тіркелуі, кіру/шығу, профиль баптаулары, тіл таңдау, аватар жүктеу функциялары бар.
- Лайк/таңдаулы механизмі арқылы интерактивті әрекет береді.
- Барлық беттерде ортақ global player арқылы ойнату тәжірибесін біріздендіреді.

Қандай мәселені шешеді?

QazSound тек CRUD деңгейімен шектелмей, толық платформа құру мәселесін шешеді: контент жариялау, тұтыну, қолданушы интеракциясы, әкімшілік аналитика және deploy-ға дайын архитектура. Сонымен қатар жүктелген аудио мен YouTube-контентті бір ойнатқышта біріктіреді.

Кімдерге арналған?

- **Музыка авторларына** - трек жариялауды жеңілдету үшін.
- **Тыңдармандарға** - заманауи интерфейсте іздеу, тыңдау, таңдаулыға қосу үшін.

- **Студенттер мен junior әзірлеушілерге** - Django-ның практикалық архитектурасын үйрену үшін.
- **Open-source қауымдастығына** - әрі қарай функционал кеңейтуге негіз ретінде.

Неге жасалды?

Жобаның негізгі мақсаты - оқу барысында алынған теорияны нақты, қолданбалы өнімге айналдыру:

- Django архитектурасын терең меңгеру;
- формалық валидация мен бизнес-логиканы дұрыс бөлу;
- фронтендте state management және асинхронды әрекеттерді іске асыру;
- deployment-ке дайын инфрақұрылым жасауды үйрену.

Ерекшелігі неде?

- Гибридті контент моделі: `UPLOAD` және `YOUTUBE`.
- YouTube URL тексеруі, нормализациясы және метадерек алу логикасы.
- `localStorage` негізінде `persistent` күй сақтайтын `global` ойнатқыш.
- `AJAX` лайк механизмі барлық компонентте синхронды жаңарады.
- Custom Django Admin аналитикалық дашборды (карталар + графиктер).
- Қазақ және орыс тіліне бейімделген интерфейс негізі.



Мүмкіндіктер

Пайдаланушы мүмкіндіктері

- **Аккаунт жүйесі:** тіркелу, кіру, шығу.
- **Профиль баптауы:** `username`, `display name`, аватар, тіл таңдау.
- **Трек жариялау:** дереккөз түрін таңдау (`UPLOAD` немесе `YOUTUBE`).
- **YouTube metadata preview:** тақырып, автор, `thumbnail`, ұзақтық мәнін автоматты толтыру.
- **Favorites жүйесі:** лайк/анлайк батырмасы, таңдаулы беті.
- **Global player:** `play/pause`, `previous/next`, ағымдағы трек күйін сақтау.
- **Іздеу және фильтр:** мәтін және жанр бойынша сұрыптау.
- **Профильдегі "Менің тректерім" бөлімі:** автор жүктеген контентті бір жерден көру.

Әкімші мүмкіндіктері

- Django Admin интерфейсін Jazzmin арқылы кеңейту және брендтеу.
- Трек карточкалары үшін `cover preview` және `rich list display`.

- Bulk action:
 - таңдалған тректерді featured ету,
 - plays_count мәнін reset ету.
- Artist жою кезіндегі қорғаныс логикасы (трегі бар авторды қате жоюды болдырмау).
- Аналитикалық дашборд:
 - жалпы қолданушылар саны,
 - жалпы тректер саны,
 - жалпы лайктар саны,
 - авторлар саны,
 - соңғы 7 күндегі жүктеу динамикасы,
 - ең көп лайк алған тректер тізімі.

Платформа деңгейіндегі мүмкіндіктер

- Мобильді және десктоп үшін бейімделген responsive интерфейс.
- Fetch + History API арқылы page-shell ішінде жылдам навигация.
- JavaScript істемей қалса да серверлік fallback логикасы бар.
- `tracks`, `users`, `interactions` модульдік құрылымы арқылы таза архитектура.
- WhiteNoise көмегімен production-та статикалық файлды сенімді беру.
- `DATABASE_URL` арқылы PostgreSQL-ге ауысу мүмкіндігі.
- JSON API endpoint-тері арқылы сыртқы интеграцияға дайын негіз.

✂ Қолданылған технологиялар

QazSound жобасы тәжірибелік және өндірістік тепе-теңдікке сүйенген технологиялық стекпен жасалған.

Технология	Неге таңдалды
Python	Түсінікті синтаксисі және серверлік логика жазудағы өнімділігі үшін.
Django	ORM, admin, auth, form, middleware сияқты дайын, сенімді блоктарды бір экожүйеде ұсынғаны үшін.
HTML5	Семантикалық құрылым мен шаблондық қайта қолдануды қамтамасыз ету үшін.
CSS3	Өз дизайн жүйесін нөлден құру, адаптивтілік пен анимация жасау үшін.

Технология	Неге таңдалды
JavaScript (Vanilla)	Player күйін басқару, AJAX лайк, динамикалық навигация, metadata fetch үшін.
Bootstrap / Tailwind	Қолданылған жоқ ; әдейі custom CSS таңдалып, UI инженериясы жеке жасалды.
SQLite	Локалды әзірлеу кезінде жылдам іске қосу және жеңіл конфигурация үшін.
PostgreSQL	DATABASE_URL арқылы production ортада масштабталатын ДҚ қолдану мүмкіндігі үшін.
Pillow	Сурет файлдарын (аватар, cover) өңдеу қолдауын беру үшін.
yt-dlp	YouTube-тан metadata және bestaudio stream URL алу үшін.
WhiteNoise	Production-та static assets беру процесін оңайлату үшін.
Gunicorn	WSGI сервері ретінде deploy кезінде сенімді жұмыс істеу үшін.
Git / GitHub	Нұсқа бақылауы, ашық код, тарих пен ынтымақтастық үшін.
VS Code	Өнімді код жазу, refactor, дебаг, файл навигациясын ыңғайлы жүргізу үшін.

Бұл стек Django-ның жылдамдығын, custom frontend басқаруын және deploy-ға дайын инфрақұрылымды тиімді біріктіреді.



Жоба архитектурасы

QazSound - модульдік Django архитектурасымен құрылған жоба. Әр доменнің өз қолданбасы бар.

Жоғары деңгейдегі құрылым

- config/ - глобалды settings, root URL, WSGI/ASGI, тесттер.
- apps/tracks/ - трек домені: модель, форма, сервис, селектор, API, YouTube логикасы.
- apps/users/ - тіркелу, кіру, профиль, баптаулар, тіл механизмі.
- apps/interactions/ - лайк/таңдаулы және playlist негізі.
- templates/ - беттер мен қайта қолданылатын компоненттер.
- static/ - CSS, JS, admin custom стильдері.
- media/ - жүктелген файлдар.

Ішкі жұмыс логикасы (қысқаша lifecycle)

1. Пайдаланушы URL ашады.
2. URL dispatcher сұранысты view-ге береді.
3. View selectors/service қабатын қолданады.
4. Form енгізуді тексереді.
5. Template HTML қайтарады.
6. JavaScript UX-ті динамикалық жаңартады.

Домен модельдері

- **Artist** - автор/орындаушы кестесі.
- **Genre** - жанр кестесі, slug автоматты генерацияланады.
- **Track** - негізгі контент:
 - source type (UPLOAD / YOUTUBE),
 - audio/cover өрістері,
 - YouTube URL/ID,
 - likes/plays метрикалары,
 - owner байланысы.
- **Like** - (user, track) unique жұбы.
- **Playlist / PlaylistItem** - болашақ playlist функционалына дайын модельдер.
- **UserProfile** - қолданушы профилінің кеңейтілген деректері.

Services / Selectors тәсілі

QazSound-та view өте "жұқа", негізгі логика бөлек қабатта:

- **selectors** - күрделі query, annotate, prefetch логикасы;
- **services** - create/update/delete бизнес операциялары;
- **forms** - енгізу валидациясы және UI-ға түсінікті қате хабарлар.

Бұл тәсіл кодты кеңейту мен тестілеуді жеңілдетеді.

YouTube ағынын өңдеу логикасы

- URL домені whitelist бойынша тексеріледі.
- Видео ID regex арқылы валидацияланады.
- Canonical URL нормализацияланады.
- Metadata (title, author, duration, thumbnail) алынады.
- Аудио stream URL тек сұраныс кезінде resolve етіледі.

Яғни серверге YouTube медиасын толық жүктеп сақтау міндетті емес.

Frontend архитектурасы

- `likes.js` - лайк күйін AJAX арқылы жаңарту.
 - `player.js` - global player, тарих стери, localStorage persistence.
 - `forms.js` - source type бойынша өрістерді көрсету/жасыру, metadata fetch.
 - `navigation.js` - толық reload жасамай page-shell ішін ауыстыру.
 - `main.js` - toast хабарламаларын басқару.
-

UI / Дизайн

QazSound интерфейсі dark modern бағытында жасалған: неон реңктер, blur-card стилі, жұмсақ көлеңке және акцентті градиенттер.

Дизайн философиясы

- **Таза интерфейс** - ақпараттық блоктар логикалық бөлінген.
- **Заманауи layout** - hero секция, карточка торы, контенттің бөлімдік құрылымы.
- **Responsive шешім** - мобильді және десктоп өлшемдеріне бейімделген.
- **Card-based UI** - тректер визуалды карточка арқылы ұсынылады.
- **Қараңғы эстетика** - фон мен акцент түс палитрасы музыкалық өнім атмосферасын береді.
- **Пайдаланушыға ыңғайлы навигация** - айқын мәзір, жылдам іздеу, нақты әрекет батырмалары.

Визуалдық ерекшеліктер

- `theme.css` ішінде CSS variables қолданылған (`--bg`, `--accent`, `--surface`).
- Фондағы анимирленген overlay интерфейске тереңдік береді.
- Track card hover эффекті интерактивтілік сезімін күшейтеді.
- Global player әр бетте бірдей көрініп, тыңдау процесін үзбейді.

UX шешімдері

- Лайк күйі бірнеше жерде бір уақытта жаңарады (карточка, detail, player).
 - YouTube metadata fetch пайдаланушының қолмен толтыру уақытын азайтады.
 - Empty state блоктары пайдаланушыға келесі қадамды нақты көрсетеді.
 - Клиенттік + серверлік валидация комбинациясы қате енгізулерді ерте ұстайды.
-

⚙ Орнату нұсқаулығы

Алдын ала талаптар

- Python 3.10+
- pip
- Git
- Виртуалды орта (venv)

1-қадам: Репозиторийді клондау

```
git clone <repository-url>
cd QazSound
```

2-қадам: Виртуалды орта құру

```
python -m venv venv
```

macOS/Linux:

```
source venv/bin/activate
```

Windows PowerShell:

```
venv\Scripts\Activate.ps1
```

3-қадам: Тәуелділіктерді орнату

```
pip install -r requirements.txt
```

4-қадам: `.env` файлын баптау

```
cp .env.example .env
```

Минималды локалды конфигурация:

```
SECRET_KEY=django-insecure-change-me
DEBUG=True
ALLOWED_HOSTS=127.0.0.1,localhost
ENABLE_YTDLP_YOUTUBE_STREAM=True
```


PostgreSQL + Firebase media сақтау үшін:

```
DATABASE_URL=postgresql://postgres:change-me@localhost:5432/qazsound
FIREBASE_STORAGE_BUCKET=your-bucket-name
FIREBASE_PROJECT_ID=your-gcp-project-id
FIREBASE_CREDENTIALS_PATH=firebase-service-account.json
FIREBASE_MEDIA_LOCATION=media
FIREBASE_QUERYSTRING_AUTH=True
```

5-қадам: Миграцияларды орындау

```
python manage.py migrate
```

6-қадам: Superuser жасау (қосымша)

```
python manage.py createsuperuser
```

7-қадам: Серверді іске қосу

```
python manage.py runserver
```

Браузерде ашыңыз: `http://127.0.0.1:8000/`

8-қадам: Static файлдарды жинау (production үшін)

```
python manage.py collectstatic --no-input
```

9-қадам: Тесттерді іске қосу

```
python manage.py test
```

Environment айнымалылар кестесі

Айнымалы	Мақсаты
<code>SECRET_KEY</code>	Django қолтаңба/қауіпсіздік кілті
<code>DEBUG</code>	Debug режимін қосу/өшіру
<code>ALLOWED_HOSTS</code>	Рұқсат етілген хосттар
<code>CSRF_TRUSTED_ORIGINS</code>	CSRF trusted origin тізімі

Айнымалы	Мақсаты
DATABASE_URL	PostgreSQL/басқа ДҚ URL қосу
DB_CONN_MAX_AGE	ДҚ қосылымының өмірлік уақыты
FIREBASE_STORAGE_BUCKET	Firebase Storage қолданатын bucket атауы
FIREBASE_PROJECT_ID	Google Cloud/Firebase project id (қосымша)
FIREBASE_CREDENTIALS_PATH	Firebase service account JSON файлына жол
FIREBASE_CREDENTIALS_JSON	Тек env қолдайтын орта үшін inline JSON
FIREBASE_MEDIA_LOCATION	Bucket ішіндегі media папкасының префиксі
FIREBASE_QUERYSTRING_AUTH	Private bucket үшін signed URL қосу/өшіру
FIREBASE_URL_EXPIRATION_SECONDS	Signed URL жарамдылық уақыты
FIREBASE_FILE_OVERWRITE	False болса файл аттарын үстінен жазбайды
FIREBASE_BLOB_CHUNK_SIZE	Үлкен аудио upload үшін chunk өлшемі
SECURE_SSL_REDIRECT	HTTPS-ке мәжбүрлі redirect
SESSION_COOKIE_SECURE	Session cookie тек HTTPS
CSRF_COOKIE_SECURE	CSRF cookie тек HTTPS
ENABLE_YTDLP_YOUTUBE_STREAM	YouTube stream функциясын қосу/өшіру
YTDLP_STREAM_FORMAT	yt-dlp формат селекторы
YTDLP_REQUEST_TIMEOUT_SECONDS	сұраныс таймауты

Deploy ескертпесі

Жобада production workflow үшін дайын файлдар бар:

- build.sh - install + migrate + collectstatic
- Procfile - Gunicorn іске қосу командасы

Бұл бұлттық PaaS ортаға орналастыруды жеңілдетеді.

Жоба құрылымы

```
QazSound/
├─ manage.py
├─ requirements.txt
├─ Procfile
├─ build.sh
├─ config/
│   ├── settings.py
│   ├── urls.py
│   ├── wsgi.py
│   ├── asgi.py
│   └─ tests/
├─ apps/
│   ├── tracks/
│   ├── users/
│   └─ interactions/
├─ templates/
│   ├── base.html
│   ├── tracks/
│   ├── users/
│   ├── interactions/
│   ├── admin/
│   ├── includes/
│   └─ components/
├─ static/
│   ├── css/
│   ├── js/
│   ├── img/
│   └─ admin/
└─ media/
```

Қалталар түсіндірмесі

- `config/` - жоба конфигурациясы және root маршруттар.
- `apps/tracks/` - тректер домені және YouTube бизнес-логикасы.
- `apps/users/` - аккаунт, профиль, тіл баптауы.
- `apps/interactions/` - лайк/таңдаулы әрекеттері.
- `templates/` - серверде рендерленетін UI қабаты.
- `static/` - стильдер, JavaScript, суреттер.
- `media/` - runtime кезінде жүктелген файлдар.

Қауіпсіздік

QazSound жобасында Django-ның стандартты қорғаныстары және доменге тән тексерулер бірге қолданылады.

Django деңгейіндегі қорғаныс

- CSRF middleware қосылған.
- Authentication middleware қолданушы сессиясын бақылайды.
- Password validator-лар қосылған (ұзындық, жиі парольдер, сандық пароль).
- Clickjacking-тен қорғау middleware-i бар.
- HTTPS/cookie қауіпсіздік параметрлері `.env` арқылы басқарылады.

CSRF және сұраныс қауіпсіздігі

- Барлық маңызды формаларда `{% csrf_token %}` қолданылады.
- AJAX POST сұраныстары CSRF header жібереді.
- `next redirect` мәні `url_has_allowed_host_and_scheme` арқылы тексеріледі.

Авторизация және рұқсат

- `@login_required` арқылы қорғалған маршруттар:
 - трек қосу/өңдеу/жою,
 - лайк қою,
 - таңдаулылар,
 - профиль және баптаулар.
- Track owner тексеруі басқа қолданушының контентін өзгертуге жол бермейді.
- Admin функциялары Django permission жүйесіне сүйенеді.

Дерек енгізу валидациясы

- Аудио файл түрі мен өлшемі тексеріледі.
- Cover image түрі/өлшемі тексеріледі.
- YouTube URL тек рұқсат етілген домендерден қабылданады.
- Видео ID қатты regex ережесімен тексеріледі.

ДҚ тұтастығы

- Like моделінде `(user, track)` unique constraint бар.
- Genre атауы мен slug бірегей.
- FK қатынастары деректер байланысын дұрыс ұстайды.



API және деректер интерфейстері

Жоба сыртқы қолданбаларға немесе фронтенд интеграцияға ыңғайлы болу үшін JSON endpoint-тер ұсынады.

Endpoint-тер

Endpoint	Әдіс	Сипаттама
/api/tracks/	GET	Тректер тізімін қайтарады (іздеу және жанр фильтрімен).
/api/tracks/<id>/	GET	Нақты трек туралы толық JSON.
/tracks/youtube/metadata/	GET	YouTube URL үшін metadata preview береді.
/tracks/<id>/stream/	GET	YouTube аудио stream URL-ін resolve етеді (redirect немесе JSON).

JSON құрамындағы негізгі өрістер

- id, title, artist
- genres
- cover_url, audio_url
- likes_count, plays_count
- source_type, youtube_url, youtube_id
- is_liked
- created_at

Маңызды ескертпе

- YouTube stream URL уақытша болуы мүмкін және upstream ережесіне тәуелді.
- plays_count өрісі рейтинг логикасында бар, автоматты аналитика pipeline-ы болашақта кеңейтіледі.



Болашақ жетілдірулер

QazSound-ты келесі деңгейге шығару үшін жоспарланған бағыттар:

1. Docker қолдауы

Орта біркелкілігін сақтау және onboarding уақытын азайту.

2. Кеңейтілген API қабаты

Мобильді және external интеграцияны жеңілдету.

3. Cloud storage интеграциясы

Медиа файлдарды масштабталатын сақтау жүйесіне шығару.

4. Playback аналитикасы

Нақты ойнату эвенттері мен статистика pipeline-ын толықтыру.

5. Тесттерді көбейту

Permission, form, service және API сценарийлерін кеңейту.

6. CI/CD автоматтандыру

Lint/test/build тексерулерін автоматты қосу.



Әзірлеуші

Turginbay Bekzat

2-курс студенті

Satbayev University, Kazakhstan

Future Full Stack Developer

Қысқа әзірлеуші тарихы

QazSound - бір айлық мақсатты инженерлік тәжірибе нәтижесі. Жоба авторға теориядан практикаға өтіп, backend архитектурасы, frontend интеракция, deploy конфигурациясы және кәсіби құжаттама жасауды бір өнім ішінде меңгеруге мүмкіндік берді.



Көзқарас

QazSound жобасының ұзақ мерзімді бағыты төрт қағидаға сүйенеді:

1. Құра отырып үйрену

Нақты өнім жасау - шынайы кәсіби өсудің ең тиімді жолы.

2. Итерациялық инновация

Алдымен мықты база, кейін үздіксіз жетілдіру.

3. Open-source мәдениеті

Кодты ашық ұсыну, пікір алу, қоғаммен бірге өсу.

4. Нақты әлемге бағытталған инженерия

Масштаб, сүйемелдеу жеңілдігі және пайдаланушы сенімі - бастапқы талап.

QazSound болашақта студенттік жобадан толыққанды ашық платформалық өнімге айналады көздейді.

★ Жобаны қолдау

Егер жоба пайдалы немесе қызық болса, келесі жолмен қолдау көрсете аласыз:

- ★ Репозиторийге **Star** қою
- 🍴 Жобаны **Fork** жасап, өз нұсқаңызды дамыту
- 🗣️ Достарыңызбен, студенттермен, қауымдастықпен бөлісу
- ✂️ Қателерді түзету, жаңа функция қосу, құжаттама жақсарту арқылы үлес қосу